Cross-Platform Performance Evaluation of Serverless Computing
A Comparative Study of AWS Lambda and Azure Functions

Anna Makridou
csdp1448@csd.uoc.gr
University of Crete, Computer Science Department
Heraklion, Greece

ABSTRACT

Serverless computing platforms like AWS Lambda and Azure Functions have gained widespread adoption, yet lack comprehensive performance comparisons using identical code and fair experimental conditions. This course report presents a microbenchmark study comparing AWS Lambda and Azure Functions across seven diverse workloads: four function benchmarks (HTML generation, image processing, file compression, and a custom matrix multiplication benchmark) and three workflow orchestration patterns (sequential, parallel, and map-reduce). Using the SeBS-Flow framework with identical code and inputs, 240 invocations were executed across both platforms, with no recorded failure flags. Results show lower mean Azure benchmark durations in all four cold-flagged and warm-flagged function groups, with 4.6–22.6× lower cold-flagged benchmark durations and up to 4.9× faster warm execution for production workloads. For workflows, AWS demonstrates consistent advantages across all three patterns: 10.6× faster for sequential workflows, 20.8× faster for parallel execution, and 15.3× faster for complex map-reduce.

CCS CONCEPTS

• **Computer systems organization** → **Cloud computing**; • **Networks** → *Network performance evaluation.*

KEYWORDS

Serverless computing, FaaS, AWS Lambda, Azure Functions, Benchmarking, Performance evaluation, Cold starts, Warm Starts, Experiments, Platform Comparison

1 INTRODUCTION

Serverless computing has revolutionized application deployment by abstracting infrastructure management and enabling automatic scaling.

Major cloud providers including Amazon Web Services (AWS Lambda), Microsoft Azure (Azure Functions) and Google Cloud Platform (GCP Cloud Functions) offer serverless platforms with distinct architectures, pricing models, and performance characteristics.

Despite widespread adoption, developers face significant challenges in selecting appropriate platforms due to limited comparative performance data. Existing studies often use platform-specific code, different input sizes or focus on single workload types, making fair comparisons difficult. Additionally, most research examines either function-level performance or workflow orchestration, but not both systematically.

This work focuses on AWS and Azure because they were the only platforms available through our educational accounts (AWS Academy and Azure for Students). These programs provided credits and temporary credentials for the experiments. The selected result files contain 240 invocations: 150 function and 90 workflow records.

1.1 Motivation

This study addresses two critical gaps in serverless performance evaluation:

(1) **Fair Comparison**: Most benchmarks use platform-specific implementations, making it unclear whether performance differences stem from platform capabilities or code optimization.
(2) **Workload Diversity**: Existing studies typically focus on single workload types, missing insights into how platforms perform across diverse application patterns.

1.2 Objectives

The research objectives are:

(1) **Validate Framework**: Confirm that SeBS-Flow, an open-source serverless benchmarking suite, works on current AWS and Azure infrastructure.
(2) **Fair Performance Comparison**: Compare AWS Lambda and Azure Functions using identical code, inputs and experimental methodology across diverse workloads.
(3) **Workload Characterization**: Analyze performance across four distinct workload types: pure compute, I/O-intensive, CPU-intensive and lightweight operations.
(4) **Workflow Evaluation**: Compare AWS Step Functions and Azure Durable Functions for serverless workflow orchestration.

2 BACKGROUND

2.1 Serverless Computing

Serverless computing abstracts infrastructure management, enabling developers to deploy code without managing servers. Key characteristics include:

- **Event-driven execution**: Functions triggered by HTTP requests, database changes and file uploads.
- **Automatic scaling**: Platform scales from zero to thousands of concurrent executions.
- **Pay-per-use pricing**: Charges based on execution time and memory, not idle resources.

- **Cold starts**: Initial invocation latency due to container initialization.

2.2 Platform Architectures

In this experiment, **AWS Lambda** used a library-based invocation path through the AWS SDK. Functions execute in Firecracker microVMs with initialization performed once per container lifecycle.

In this experiment, **Azure Functions** used HTTP endpoints. This invocation path includes network communication; its separate contribution to the observed performance gap was not measured.

3 EXPERIMENTS

3.1 Experimental Setup

3.1.1 Infrastructure Configuration. Table 1 shows the experimental configuration. AWS us-east-1 was used for AWS Lambda. Azure francecentral was selected for Azure Functions due to Azure for Students account restrictions, limiting deployment to five specific EU regions (francecentral, polandcentral, norwayeast, switzerlandnorth, germanywestcentral), preventing deployment to US regions. The standard workloads ran Python 3.8; the custom Azure matrix benchmark used Python 3.11. The project specified 128–256 MB memory settings. Different regions, invocation paths and runtime versions limit the fairness of the comparison and prevent attributing differences to a single platform mechanism.

Table 1: Experimental Configuration

Parameter	AWS Lambda	Azure Functions
Runtime	Python 3.8	Python 3.8/3.11
Region	us-east-1	francecentral
Memory	128–256 MB	128–256 MB
Trigger Type	Library (direct)	HTTP (network)
Repetitions	15–30	15–30
Success Rate	100%	100%

3.1.2 Workflow Infrastructure. For workflow benchmarks, the following infrastructure was deployed:

- **AWS**: EC2 t2.micro instance (us-east-1) running Redis 7.0.15 for cross-function coordination.
- **Azure**: Standard B1s VM (francecentral) running Redis 7.0.15
- **Orchestration**: AWS Step Functions (library trigger) and Azure Durable Functions. (HTTP trigger)

3.2 Benchmark Selection

Seven benchmarks were selected representing diverse serverless workload patterns.

3.2.1 Function Benchmarks.

(1) **110.dynamic-html**: Pure compute workload generating HTML. Tests CPU performance with minimal I/O.
(2) **210.thumbnailer**: I/O-intensive image processing workload. Reads images from cloud storage, resizes and writes results back. Tests storage I/O and image processing performance.
(3) **311.compression**: CPU-intensive file compression using gzip. Compresses large text files to test sustained CPU utilization.
(4) **050.matrix-multiply**: Custom lightweight benchmark implemented to test framework functionality and minimal workload performance. Performs matrix multiplication, with mean warm benchmark durations of 1.349 ms on AWS and 0.392 ms on Azure in these runs.

3.2.2 Workflow Benchmarks.

(1) **620.func-invo**: Sequential function invocation chain. Measures orchestration latency and coordination overhead for linear sequential workflows.
(2) **630.parallel-sleep**: Parallel fan-out pattern where generator function spawns N parallel tasks. Tests concurrent execution and synchronization.
(3) **660.map-reduce**: Complete map-reduce workflow implementing a full distributed data processing pipeline. Tests orchestration efficiency for complex multi-stage workflows with dynamic parallelism.

3.3 Collected Metrics

For each invocation, the following metrics were measured:

- **Cold-flagged benchmark duration**: Mean `times.benchmark` for invocations marked by `stats.cold_start`; this does not isolate initialization overhead.
- **Warm execution time**: Mean `times.benchmark` for invocations without the cold-start flag.
- **Success rate**: Percentage of successful completions.
- **Workflow latency**: Client-observed elapsed time from invocation to completion, recorded in `times.client`. This includes the client-to-platform invocation and completion path; it is not isolated orchestration overhead.
- **Cold start count**: Number of cold starts across repetitions.

Raw timing fields are in microseconds. Function values are divided by 1,000 to obtain milliseconds; workflow values below use client latency and the same conversion. Each function has one AWS and two Azure cold-flagged samples. Warm sample counts are 29/28 for HTML and 14/13 for the other functions (AWS/Azure). Each workflow has 15 samples per platform. The 240 records come from the eight dated function files and six `experiments_620/630/660` files under `MY_RESEARCH_OUTPUT/aws_results` and `azure_results`.

3.4 Experimental Protocol

Each benchmark execution followed this protocol:

(1) Deploy function code to platform using SeBS-Flow automated deployment
(2) Execute 15–30 invocations with identical inputs
(3) Collect metrics through platform APIs (AWS CloudWatch, Azure Application Insights)
(4) Validate 100% success rate before analysis
(5) Export results to JSON for offline analysis

4 CHALLENGES AND FAILURES

Significant technical challenges were encountered during the setup and reproduction of this paper. These obstacles, while initially hindering progress, provided valuable insights into the practical complexities of cross-platform serverless benchmarking on educational cloud accounts.

4.1 AWS Academy SSL Certificate Issues

Problem: Certificate configuration errors in the experimental AWS Academy setup affected boto3 client connections, causing `ValueError: empty or no certificate` errors. All AWS API calls failed, preventing benchmark execution.

Solution: A custom SSL error handling module was created that:

(1) Attempts standard SSL verification first.
(2) Falls back to disabled verification for AWS Academy.
(3) Keeps verification enabled on the standard path; the disabled-verification fallback weakens endpoint authentication and is unsuitable for production.

This modification required updates to AWS modules including S3, CloudWatch, and Lambda client creation code.

4.2 Temporary Credential Handling

Problem: AWS Academy uses temporary credentials with session tokens, but the local implementation needed changes to propagate session tokens.

Solution: Enhanced credential handling was implemented to detect and use the `AWS_SESSION_TOKEN` environment variable. This required:

(1) Modifying configuration loading logic.
(2) Updating all boto3 client instantiations.
(3) Adding session token parameter propagation.

4.3 Azure Durable Functions Complexity

Problem: Azure workflow benchmarks require Azure Durable Functions, which are significantly more complex than AWS Step Functions:

(1) Require specific Azure Storage configuration.
(2) Need HTTP trigger setup.
(3) Involve more complex orchestration runtime.

Initial Challenges: Deployment and configuration of Azure Durable Functions required additional setup steps compared to AWS, including:

(1) Configuring Azure Storage accounts for orchestration state.
(2) Setting up HTTP endpoints for workflow invocation.
(3) Managing orchestration runtime dependencies.

Outcome: Despite the additional complexity, all 45 recorded invocations across the three Azure workflows completed without failure flags. This establishes completion of these runs, rather than production readiness or general reliability.

Lessons Learned: Cross-platform workflow portability requires understanding fundamental architectural differences in orchestration systems. Experimental results show lower mean client-observed latency for AWS across all tested workflow patterns (10-21× lower),

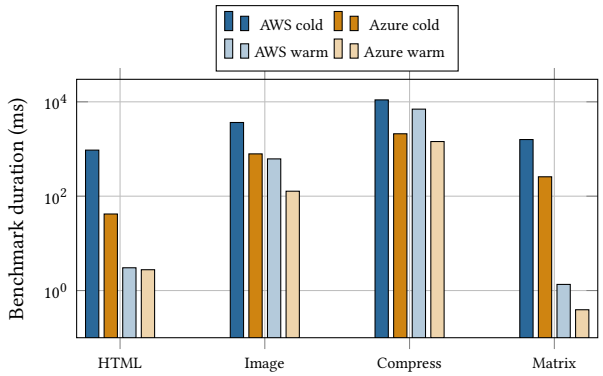


Figure 1: Microbenchmark performance comparison with log-scale Y-axis showing execution times. Azure (orange) has lower mean benchmark durations than AWS (blue) in the selected cold and warm groups. Cold-group counts are one AWS and two Azure samples per function; these are not isolated initialization timings.

while Azure means were 7-15 seconds. The experiments do not isolate trigger or coordination overhead as the cause.

5 IMPLEMENTATION

5.1 SeBS-Flow Modifications

SeBS-Flow was extended with:

(1) **SSL Error Handling**: Custom `ssl_fix.py` module providing graceful fallback for AWS Academy environments.
(2) **Session Token Support**: Enhanced credential handling for temporary AWS credentials.
(3) **Custom Benchmark Implementation**: Created 050.matrix-multiply benchmark to test framework functionality with minimal workload. A complete benchmark structure including function code, configuration files, and deployment scripts was implemented following SeBS-Flow conventions.
(4) **Automated Deployment Scripts**: Created `run_aws_benchmark.sh` and `run_azure_benchmark.sh` for reproducible execution.

6 EVALUATION

6.1 Function Benchmark Performance

Table 2 presents complete performance results for all four function benchmarks, comparing cold-flagged and warm-flagged benchmark durations between AWS Lambda and Azure Functions. Each benchmark represents a distinct workload pattern (compute, I/O, CPU-intensive, custom lightweight), enabling comprehensive cross-platform performance characterization.

6.1.1 Cold Start Analysis. Azure had lower mean cold-flagged benchmark durations across the production workloads in these runs (4.6–22.6×, see Figure 1). Possible explanations, not isolated by this experiment, include:

- More efficient container initialization.
- Faster dependency loading for Python packages.
- Better optimized runtime environment.

Table 2: Function Benchmark Performance Comparison

Benchmark	Type	AWS Cold	Azure Cold	AWS Warm	Azure Warm	Cold Winner	Warm Winner
110.dynamic-html	Compute	947 ms	42 ms	3.038 ms	2.763 ms	Azure (22.6×)	Azure (1.1×)
210.thumbnailer	I/O	3,648 ms	789 ms	617 ms	128 ms	Azure (4.6×)	Azure (4.8×)
311.compression	CPU	10,987 ms	2,103 ms	7,000 ms	1,440 ms	Azure (5.2×)	Azure (4.9×)
050.matrix-multiply	Custom	1,587 ms	258 ms	1.349 ms	0.392 ms	Azure (6.2×)	Azure (3.4×)

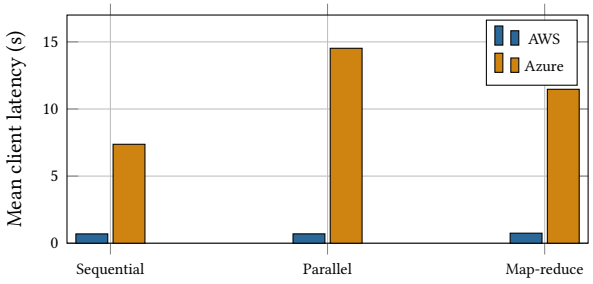


Figure 2: Client-observed workflow latency comparison. Bar chart shows mean latency for three workflow patterns. AWS (blue) consistently outperforms Azure (orange) across all workflow types: 10.6× faster for sequential (620), 20.8× faster for parallel (630), and 15.3× faster for map-reduce (660).

Furthermore, Azure has 6.2× lower mean cold-flagged benchmark duration for the custom benchmark within this dataset. With only one AWS and two Azure cold-flagged samples per function, broader cold-start distributions remain untested.

6.1.2 Warm Execution Analysis. Azure had lower mean warm benchmark duration in all four comparisons (1.1–4.9×). Potential explanations, not independently measured here, include:

- Superior CPU performance for compute workloads
- More efficient I/O for storage operations
- Better-optimized Python runtime

The performance gap widens for CPU-intensive workloads (311.compression: 4.9×), but these measurements do not establish whether hardware, CPU scheduling or another configuration difference caused the gap. These benchmark durations do not imply lower user-observed response times in every case: mean client latency across HTML invocations was 268.3 ms on AWS versus 335.4 ms on Azure, and matrix client latency was 354.4 ms versus 459.3 ms. Workflow latency reveals a different performance landscape.

6.2 Workflow Orchestration Performance

Table 3 presents workflow orchestration performance comparing AWS Step Functions and Azure Durable Functions across three patterns. AWS demonstrates consistent advantages across all patterns, with speedups ranging from 10.6× to 20.8× faster than Azure.

6.2.1 Orchestration Overhead. The results demonstrate dramatic performance differences between platforms. AWS achieves sub-second mean client latency across all three workflow patterns (693.5-747.9ms),

Table 3: Workflow Orchestration Performance

Workflow	AWS	Azure	Winner
620.func-invo	693.5 ms	7,370.9 ms	AWS (10.6×)
630.parallel-sleep	697.2 ms	14,523.6 ms	AWS (20.8×)
660.map-reduce	747.9 ms	11,468.6 ms	AWS (15.3×)

while Azure experiences 7-15 seconds of total latency even for simple sequential chains. Different regions and invocation paths prevent isolating the cause of this performance gap.

Results reveal consistent orchestration performance advantages for AWS across all workflow patterns (see Figure 2 and Figure 3). The uploaded timing fields do not support a comparable critical-path/overhead decomposition: AWS workflow `times.benchmark` is zero, while the Azure field has a different measurement scope.

AWS Step Functions wins all 3 workflows:

- 10.6× faster for sequential workflow. (620.func-invo: 693.5ms vs 7,370.9ms)
- 20.8× faster for parallel workflow. (630.parallel-sleep: 697.2ms vs 14,523.6ms)
- 15.3× faster for complex map-reduce. (660.map-reduce: 747.9ms vs 11,468.6ms)
- AWS used the library invocation path; its separate contribution to latency was not measured.
- State-transition and coordination costs were not independently isolated.

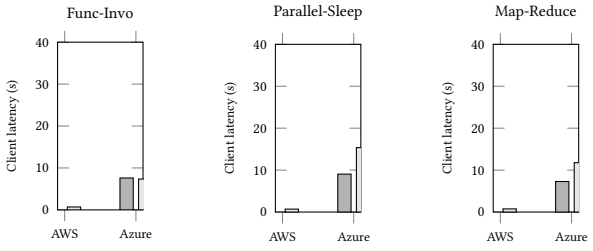


Figure 3: Mean client-observed workflow latency grouped by the recorded cold/warm flag. Dark bars show cold-flagged means and light bars show warm-flagged means; separate bars are not added. AWS has no cold-flagged samples; Azure has 1, 2 and 1, respectively. Small cold groups do not establish cold-start distributions. Sub-second AWS means do not imply that every invocation was sub-second.

- All three mean client latencies were below 750ms; individual observations were not always below this threshold.

Azure Durable Functions performance limitations:

- The tested HTTP-based deployment had higher mean client latency for all workflow patterns.
- 7-15 second total latency even for simple sequential and parallel workflows.
- The available timing fields do not establish the fraction attributable to overhead.
- Additional controlled measurements are needed to assess the effect of HTTP-based coordination.

Critical Path Analysis

The selected records do not contain a complete critical-path measurement on both platforms. Consequently, client latency cannot be reliably decomposed into function execution and orchestration overhead. Additional instrumentation with comparable timing boundaries is needed for this analysis.

7 RESULTS

7.1 Key Findings

(1) **Workload separation**: Azure had lower mean function benchmark durations, while AWS had lower mean workflow client latency in the tested configurations.
(2) **Workflow latency**: Azure means were 7–15s and AWS means were below 750ms. These are client-observed timings, not isolated orchestration delays.
(3) **Cold groups and recorded completion**: Azure had 4.6–22.6× lower mean cold-flagged benchmark durations; all 240 records had no failure flags. Small cold samples and recorded completion do not establish general cold-start behavior or reliability.

7.2 Framework Validation

SeBS-Flow was successfully validated on AWS and Azure infrastructure, confirming:

- Cross-platform deployment automation functional
- Benchmark implementations portable across platforms
- Timing fields collected for the selected runs, subject to the scope limitations above
- Framework suitable for comparative performance studies

However, limitations remain: predefined input sizes limit exact artifact replication, regions and runtime versions differ, cold-flagged sample counts are small, and the SSL-verification workaround weakens endpoint authentication.

8 RELATED WORK

Our results align with the SeBS-Flow evaluation [1] regarding function-level performance, where Azure demonstrated superior cold start latency and consistent CPU allocation. Like Schmid et al., we observed Azure's significant advantages for individual function execution across diverse workload types.

However, our workflow orchestration findings diverge substantially. While Schmid et al. reported Azure Durable Functions achieving competitive performance for certain workflow patterns [1], our experiments show AWS dominating all three workflow categories with 10.6–20.8× speedups. Two potential explanations, not isolated by these experiments, are: (1) our Azure deployment to *francecentral* due to educational account restrictions may have experienced higher orchestration overhead compared to other Azure regions and (2) our HTTP-based trigger configuration for Azure workflows consistently exhibited 7–15 seconds of mean client latency, suggesting sensitivity to regional infrastructure and configuration choices. These findings highlight the importance of region selection and trigger configuration in workflow orchestration benchmarks.

9 CONCLUSIONS

This study provides comprehensive cross-platform performance evaluation of AWS Lambda and Azure Functions using identical code across seven diverse benchmarks. The findings reveal clear performance separation by workload type: Azure excels at individual function execution with up to 22.6× lower mean cold-flagged benchmark durations, while AWS dominates workflow orchestration with 10-21× lower mean client latency across sequential, parallel, and map-reduce patterns.

SeBS-Flow was exercised on AWS and Azure in December 2025, with no recorded failure flags across 240 invocations, while discovering important limitations around educational cloud environments and workflow orchestration complexity. The documented challenges including SSL certificate handling and temporary credentials provide valuable guidance for future serverless benchmarking research.

REFERENCES

[1] Larissa Schmid, Marcin Copik, Alexandru Calotoiu, Laurin Brandner, Anne Koziolek, and Torsten Hoefler. 2025. SeBS-Flow: Benchmarking Serverless Cloud Function Workflows. In *Twentieth European Conference on Computer Systems (EuroSys '25), March 30-April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 19 pages. https://doi.org/10.1145/3689031.3717465